

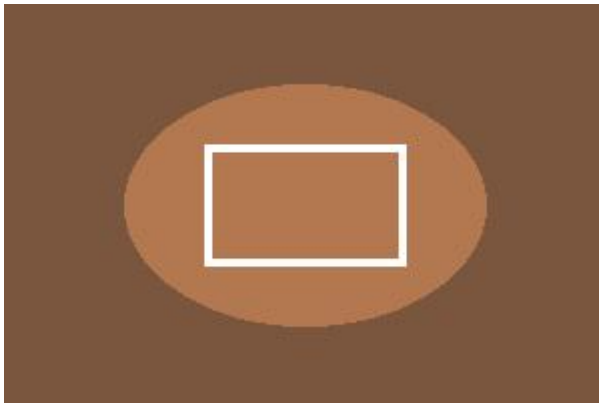
```
In [1]: import os
        from PIL import Image, ImageDraw

        if not os.path.exists("foto.jpg"):
            im = Image.new("RGB", (300, 200), (122, 86, 62))
            d = ImageDraw.Draw(im)
            d.ellipse([60, 40, 240, 160], fill=(180, 120, 80))
            d.rectangle([100, 70, 200, 130], outline=(255, 255, 255), width=4)
            im.save("foto.jpg", "JPEG")
            print("Создан foto.jpg")
```

Создан foto.jpg

```
In [2]: from PIL import Image

        img = Image.open("foto.jpg")
        # В Jupyter вместо img.show() выводим изображение в ячейку:
        display(img)
```



```
In [3]: ## Задание 1. Получение и изменение цвета пиксела

        ### 1.1. Метод `load()`
        #Сначала смотрим цветовую модель изображения, затем читаем и меняем цвет
```

```
In [4]: from PIL import Image

        img = Image.open("foto.jpg")
        print("Режим:", img.mode)

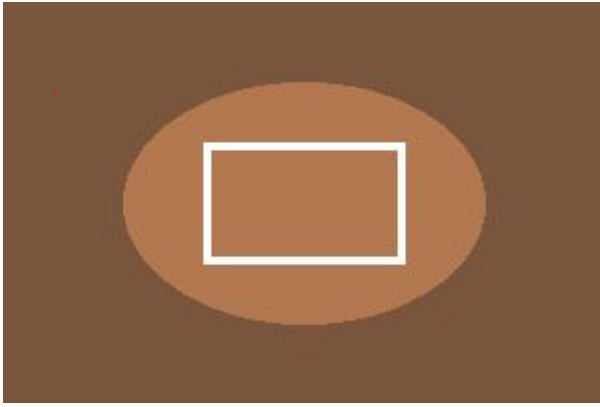
        obj = img.load()
        print("Цвет пикселя [25, 45]:", obj[25, 45])

        # Задаём цвет пикселя (красный)
        obj[25, 45] = (255, 0, 0)
        print("После изменения:", obj[25, 45])
        display(img)
```

Режим: RGB

Цвет пикселя [25, 45]: (122, 86, 62)

После изменения: (255, 0, 0)



In [5]: `### 1.2. Методы `getpixel()` и `putpixel()``

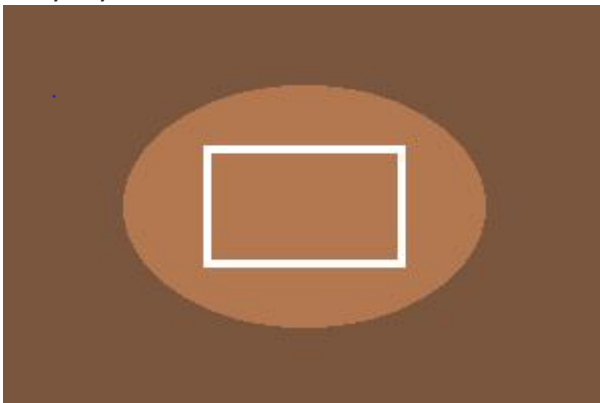
```
In [6]: # Получаем цвет пикселя
print(img.getpixel((25, 45)))

# Изменяем цвет пикселя (синий)
img.putpixel((25, 45), (0, 0, 255))

# Получаем цвет пикселя
print(img.getpixel((25, 45)))
display(img)
```

(255, 0, 0)

(0, 0, 255)



In [7]: `## Задание 2. Переход от одной цветовой модели к другой`

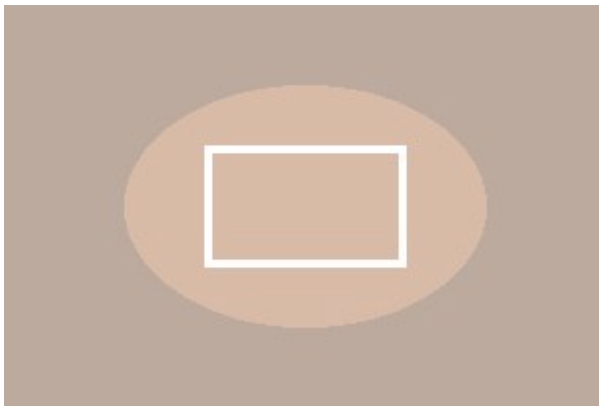
```
### 2.1. RGB → RGBA через `split()` и `merge()`
# `split()` возвращает каналы изображения кортежем; `merge(<Режим>, <Канал
```

```
In [8]: img = Image.open("foto.jpg")
print("mode:", img.mode)

R, G, B = img.split()
mask = Image.new("L", img.size, 128) # альфа-канал (полупрозрачность)
img2 = Image.merge("RGBA", (R, G, B, mask))
print("mode:", img2.mode)
display(img2)
```

mode: RGB

mode: RGBA



In [9]: `### 2.2. RGB → RGBA через `convert()```

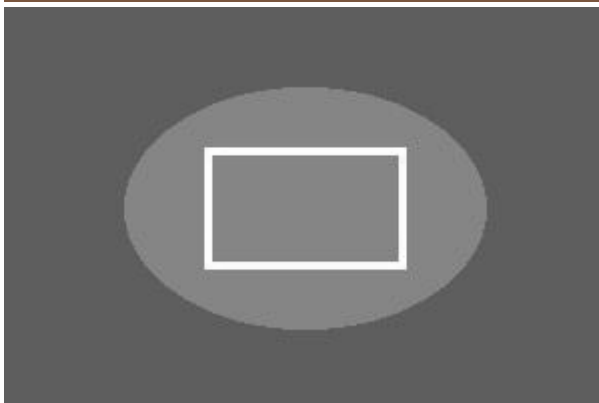
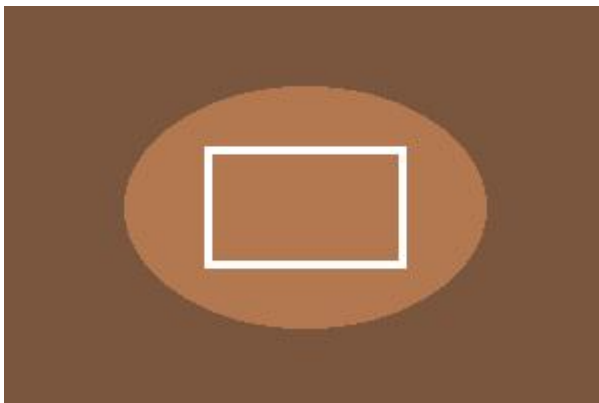
```
In [10]: img = Image.open("foto.jpg")
print(img.mode)

img2 = img.convert("RGBA")
print(img2.mode)
display(img2)

# Прочитать изображение и конвертировать в градации серого:
pil_im = Image.open("foto.jpg").convert("L")
display(pil_im)
```

RGB

RGBA



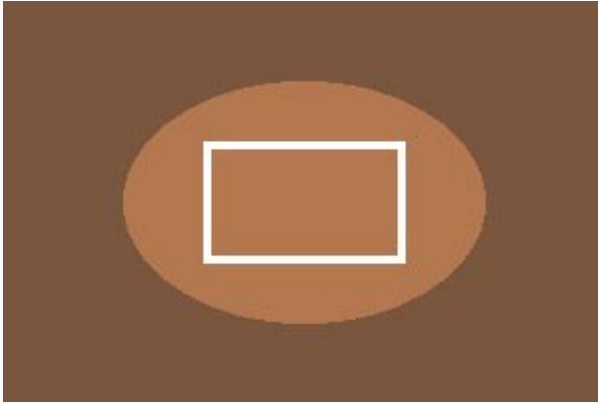
In [11]: `### 2.3. RGB → P (смешивание цветов + адаптивная палитра в 128 цветов)`

```
In [12]: img = Image.open("foto.jpg")
print(img.mode)

img2 = img.convert("P", None, Image.FLOYDSTEINBERG, Image.ADAPTIVE, 128)
```

```
print(img2.mode)
display(img2)
```

RGB
P



In [13]: *## Задание 3. Сохранение изображения*
#Сохраняем изображение в текущую папку в разных форматах.

```
In [14]: img = Image.open("foto.jpg")

# в формате JPEG
img.save("tmp.jpg")

# в формате BMP
img.save("tmp.bmp", "BMP")

# через файловый объект
f = open("tmp2.bmp", "wb")
img.save(f, "BMP")
f.close()

print("Сохранены файлы:", [n for n in os.listdir(".") if n.startswith("tm

Сохранены файлы: ['tmp.jpg', 'tmp2.bmp', 'tmp.bmp']
```

In [15]: *## Задание 4. Создание нового изображения*
#Разные способы задать цвет: кортеж RGB, имя цвета, HEX, `rgb(...)`, проц

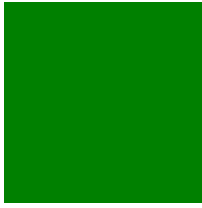
```
In [16]: # 1. Чёрный квадрат
display(Image.new("RGB", (100, 100)))
```



```
In [17]: # 2. Красный квадрат (кортеж RGB)
display(Image.new("RGB", (100, 100), (255, 0, 0)))
```



```
In [18]: # 3. Зелёный квадрат (имя цвета)
display(Image.new("RGB", (100, 100), "green"))
```



```
In [19]: # 4. Красный квадрат (HEX)
display(Image.new("RGB", (100, 100), "#f00"))
```



```
In [20]: # 5. Белый квадрат
display(Image.new("RGB", (100, 100), "white"))
```

```
In [21]: # 6. Серый квадрат
display(Image.new("RGB", (320, 240), "silver"))
```



```
In [22]: # 7. Цветной прямоугольник через rgb(...)
display(Image.new("RGB", (320, 240), "rgb(205, 100, 200)"))
```



```
In [23]: # 8. Цветной прямоугольник (каналы в процентах)
display(Image.new("RGB", (320, 240), "rgb(10%, 100%, 40%)"))
```

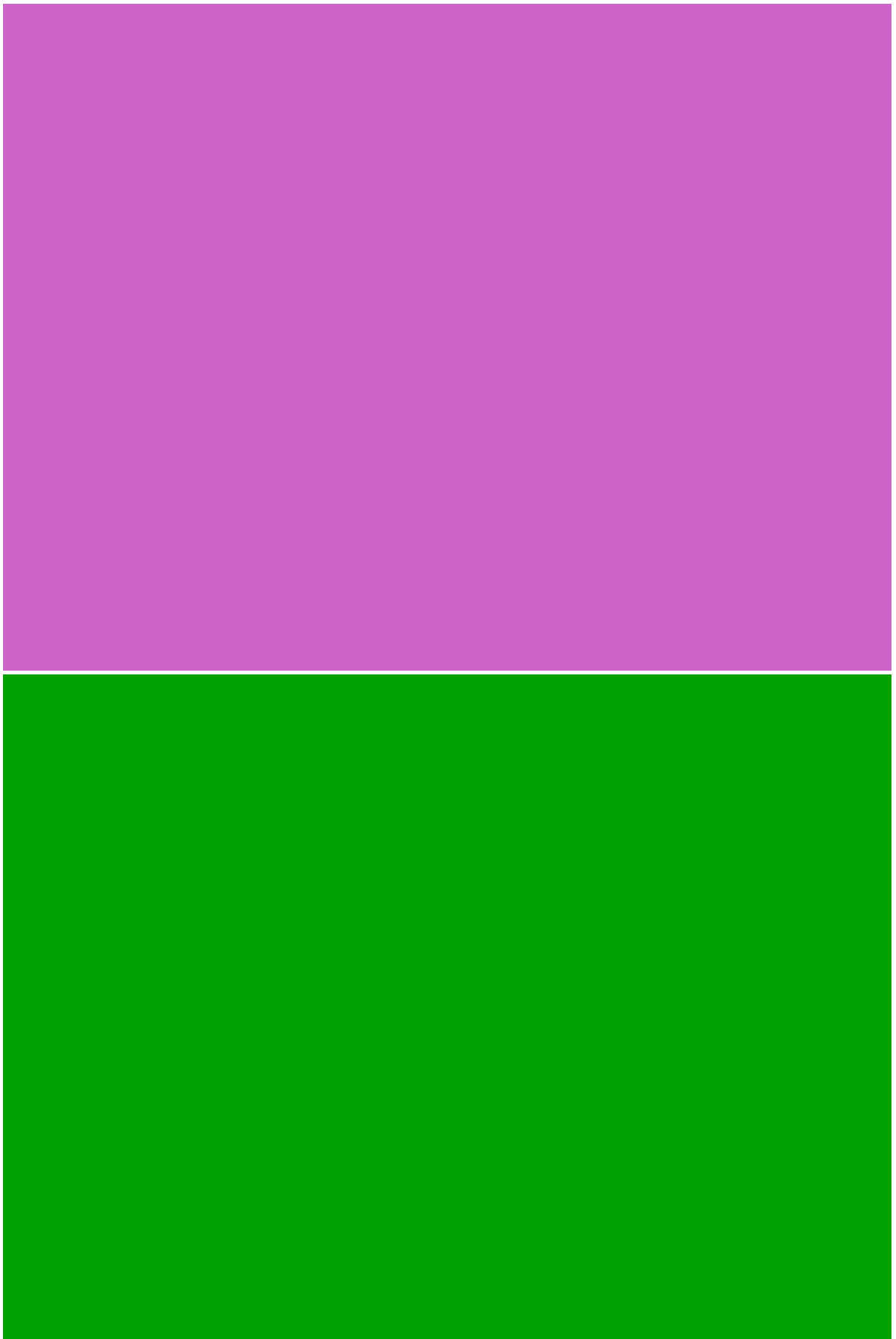


```
In [24]: ### 4.9. Создать прямоугольник, затем перекрасить его попиксельно
#(в методичке `xrange` – это Python 2; в Python 3 используем `range`)
```

```
In [25]: img = Image.new("RGB", (640, 480), "rgb(205, 100, 200)") # сиреневый
display(img)

for x in range(640):
    for y in range(480):
        img.putpixel((x, y), (0, 160, 0))

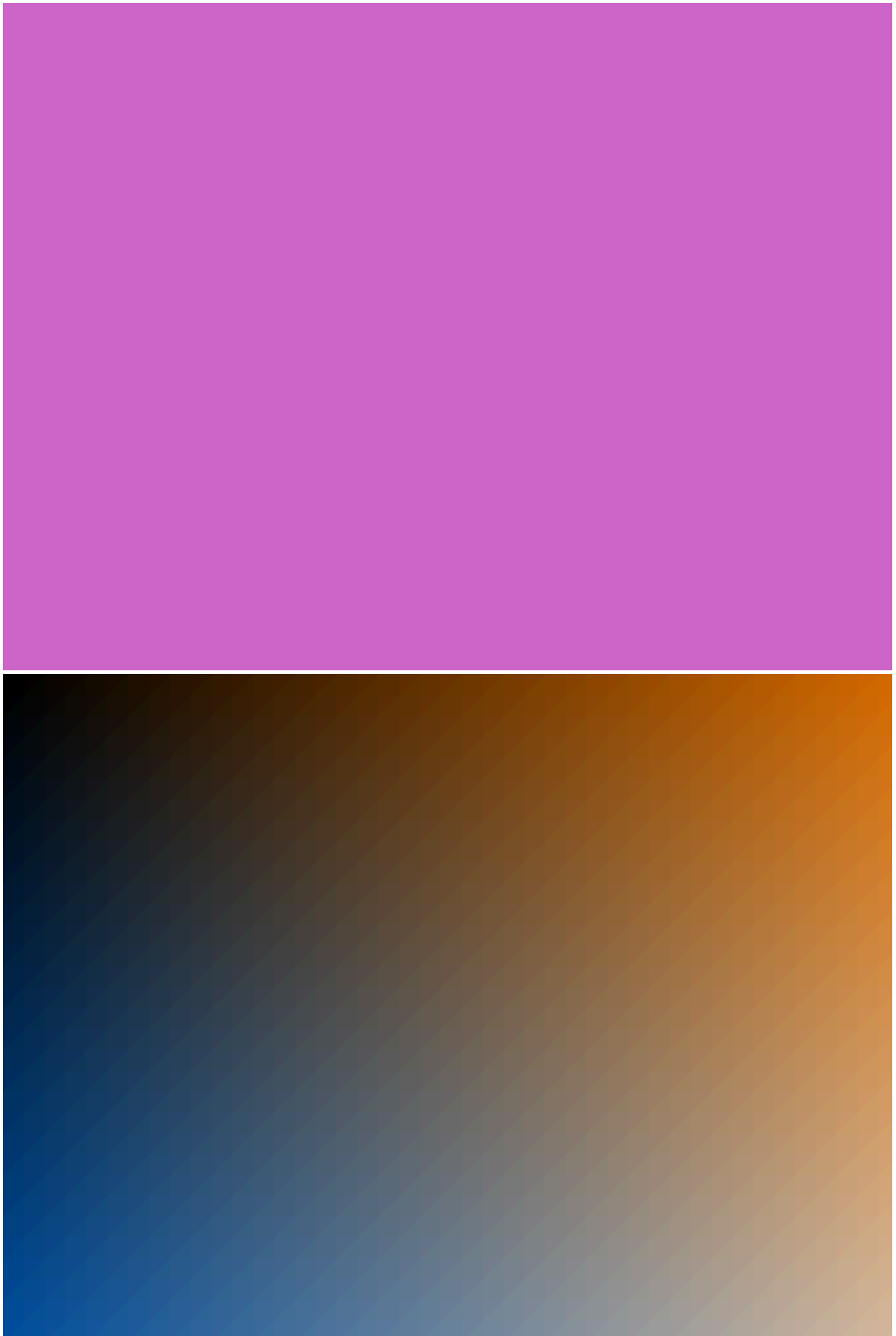
img.save("okno.png", "PNG")
display(img) # зелёный
```



```
In [26]: ### 4.10. Прямоугольник с функциональной раскраской  
#Цвет каждого пикселя зависит от его координат. В Python 3 используем цел
```

```
In [27]: img = Image.new("RGB", (640, 480), "rgb(205, 100, 200)") # сиреневый  
display(img)
```

```
for x in range(640):  
    for y in range(480):  
        img.putpixel((x, y), (x // 3, (x + y) // 6, y // 3))  
  
img.save("okno.png", "PNG")  
display(img) # функциональная раскраска (градиент)
```



In []: